

APLIKASI GROSIR BUKU BERBASIS WEB DENGAN MODUL PEMBAYARAN HUTANG

Disusun guna untuk memenuhi Tugas Akhir Ujias Akhir Semester

pada mata kuliah Sistem Basis Data

Dosen Pengampu: Dani Rohpandi, M.Kom



Disusun Oleh Kelompok 5:

Muhammad Rafi	: 251110008
Ahmad Zaky Mikail	: 251110199
Ayu Satya Aryani	: 251110059
Kireina Syifa Nabila	: 251110066
Riva Isna Salsabila	: 251110046
Mufti Anggraeni	: 251110093

**SEKOLAH TINGGI MANAJEMEN INFORMATIKA
STMIK MARDIRA INDONESIA
BANDUNG JAWA BARAT
2026**

KATA PENGANTAR

Puji dan syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan yang berjudul "Aplikasi Grosir Buku Berbasis Web dengan Modul Pembayaran Hutang". Laporan ini disusun untuk memenuhi tugas Ujian Akhir Semester (UAS) mata kuliah Sistem Basis Data pada Program Studi S1 Teknik Informatika, Sekolah Tinggi Manajemen dan Informatika Mardira Indonesia.

Penyusunan laporan ini didasarkan pada hasil analisis proses bisnis, rancangan database, implementasi migration dan model, analisis controller, serta evaluasi sistem yang dilakukan secara bertahap setiap minggunya. Aplikasi ini dikembangkan untuk membantu pencatatan transaksi hutang kepada supplier secara terstruktur, efisien, dan sesuai kebutuhan operasional usaha grosir buku.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu serta rekan-rekan sangat diharapkan agar laporan ini dapat diperbaiki pada kesempatan berikutnya. Penulis mengucapkan terima kasih kepada Bapak Dani Rohpandi, M.Kom. selaku dosen mata kuliah Sistem Basis Data yang telah memberikan arahan dan bimbingan dalam penyusunan laporan ini.

Semoga laporan ini bermanfaat bagi penulis khususnya dan pembaca umumnya sebagai bahan pengetahuan dan referensi dalam memahami konsep dasar serta penerapan sistem basis data relasional.

Bandung, Juli 2026
Kelompok 5

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR ISI	4
BAB I	6
1.1 Latar Belakang	6
1.2 Rumusan Masalah	6
1.3 Tujuan Laporan	6
1.4 Manfaat Laporan	7
1.5 Batasan Masalah	7
BAB II	8
2.1 Tinjauan Pustaka	8
2.2 Gambaran Umum Sistem	8
2.3 Permasalahan yang Dihadapi	8
2.4 Solusi yang Ditawarkan Sistem	9
2.5 Proses Pengembangan Setiap Minggu	9
BAB III	11
3.1 Metode Pengembangan Sistem	11
3.2 Objek Penelitian	11
3.3 Metode Pengumpulan Data	11
3.4 Analisis Kebutuhan Sistem	11
3.4.1 Kebutuhan Fungsional	11
3.4.2 Kebutuhan Nonfungsional	12
3.5 Perancangan Sistem	12
3.5.1 Aturan Bisnis	12
3.5.2 Struktur Database Modul Pembayaran Hutang	12
3.5.3 Relasi Antar Entitas	14
3.5.4 Alur Proses Bisnis	14
3.5.5 Perancangan Antarmuka	15
BAB IV	17
4.1 Implementasi Database (Migration)	17
A. Tabel suppliers	17
B. Tabel pembelians	18
C. Tabel detail_pembelians	18
D. Tabel hutangs	19
E. Tabel pembayaran_hutangs	20
4.2 Implementasi Model Eloquent	22
A. Model Supplier	22
B. Model Pembelian	22
C. Model Hutang	23
D. Model PembayaranHutang	23
4.3 Implementasi Controller	25

4.4 Evaluasi Implementasi	26
A. Kelebihan Sistem	26
B. Kekurangan / Kelemahan Sistem Saat Ini	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong banyak usaha untuk beralih dari pencatatan konvensional ke sistem informasi berbasis web. Pada operasional usaha grosir buku, proses transaksi pembelian dari supplier merupakan tulang punggung aktivitas bisnis karena berhubungan langsung dengan pencatatan barang masuk, pengelolaan kewajiban pembayaran (hutang), dan penjagaan hubungan baik dengan supplier atau penerbit. Jika proses tersebut masih dilakukan secara manual menggunakan nota kertas atau buku besar, maka risiko terjadinya kesalahan pencatatan (human error), kehilangan data transaksi, lambatnya pencarian riwayat hutang, dan keterlambatan pembayaran akan semakin besar.

Aplikasi grosir buku berbasis web dirancang untuk membantu proses operasional pembelian dan pembayaran hutang agar lebih cepat, rapi, dan akurat. Dalam arsitektur aplikasi ini, modul pembayaran hutang menjadi bagian paling inti karena karakteristik transaksi pembelian pada usaha grosir tidak selalu diselesaikan secara tunai. Dalam praktiknya, toko dapat membeli buku dari supplier secara kredit (tempo), sehingga sistem harus mampu membentuk kartu hutang secara otomatis saat transaksi disimpan dan mencatat setiap cicilan pembayaran yang dilakukan.

Selain itu, fluktuasi harga beli buku yang dinamis menuntut sistem untuk memiliki mekanisme pengamanan data finansial. Sistem perlu menyimpan harga snapshot pada saat transaksi berlangsung ke dalam tabel detail pembelian, sehingga histori transaksi dan laporan hutang tetap akurat meskipun harga barang di master data berubah di kemudian hari. Melalui implementasi basis data relasional yang terintegrasi, seluruh informasi transaksi dapat disimpan secara konsisten, aman, dan mudah ditelusuri kembali untuk keperluan audit internal toko.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam laporan ini adalah:

1. Bagaimana proses perancangan aplikasi grosir buku berbasis web dengan modul pembayaran hutang yang adaptif terhadap kebutuhan toko?
2. Bagaimana struktur database dan skema relasi antar entitas dirancang untuk mendukung pemisahan alur transaksi pembelian tunai dan kredit?
3. Bagaimana implementasi teknis migration, model Eloquent, dan resource controller menggunakan framework Laravel pada sistem?
4. Bagaimana logika program memproses pembentukan kartu hutang secara otomatis dan terintegrasi?
5. Bagaimana pencatatan kartu pembayaran hutang dan pembaruan status pelunasan dikelola dalam aplikasi?
6. Bagaimana sistem menangani potensi masalah integritas data hutang dan histori pembayaran pada level backend?

1.3 Tujuan Laporan

Adapun tujuan penyusunan laporan ini adalah:

1. Menjelaskan proses analisis dan perancangan aplikasi grosir buku berbasis web dari tahap perencanaan hingga evaluasi.
2. Menjabarkan skema tabel database, foreign key constraint, dan relasi antar tabel pada modul pembayaran hutang secara detail.
3. Mendokumentasikan implementasi baris kode (source code) untuk komponen migration, model, dan controller menggunakan Laravel.
4. Menguraikan alur logika bisnis pembentukan hutang serta pencatatan pembayaran hutang di dalam sistem.
5. Menyajikan hasil evaluasi objektif mengenai kelebihan, kekurangan, dan rencana tindak lanjut optimasi modul pembayaran hutang.

1.4 Manfaat Laporan

Manfaat yang diharapkan dari penulisan laporan dan pembangunan sistem ini adalah:

1. Bagi Operasional Toko: Membantu staf pembelian mencatat transaksi pembelian dan pembentukan hutang dengan lebih cepat dan rapi.
2. Bagi Manajemen Keuangan: Memudahkan pemantauan posisi hutang toko kepada tiap supplier serta pelacakan tanggal jatuh tempo secara otomatis.
3. Bagi Hubungan Supplier: Menjaga ketepatan waktu pembayaran sehingga kepercayaan dan kerja sama dengan supplier tetap terjaga.
4. Bagi Akademik: Menjadi dokumen referensi dan bukti penerapan implementasi sistem basis data relasional terintegrasi menggunakan framework MVC modern.

1.5 Batasan Masalah

Agar pembahasan dalam laporan ini lebih terarah dan spesifik, maka ruang lingkup batasan masalah ditentukan sebagai berikut:

1. Fokus pembahasan hanya dilakukan pada modul pembayaran hutang beserta fitur penunjangnya pada aplikasi GrosirBuku.
2. Pengelolaan data terbatas pada transaksi pembelian dari supplier, pembentukan kartu hutang, dan pencatatan cicilan pembayaran hutang.
3. Framework yang digunakan dalam implementasi backend adalah Laravel dengan database management system (DBMS) MySQL.
4. Analisis performa dan arsitektur didasarkan pada riwayat proses pengembangan mingguan dan repositori kode terkini.

BAB II

TINJAUAN PUSTAKA DAN GAMBARAN UMUM SISTEM

2.1 Tinjauan Pustaka

Sistem informasi pembelian dan hutang dirancang untuk mengotomatisasi seluruh alur pencatatan transaksi pembelian barang dari supplier, pengelolaan kewajiban pembayaran (hutang usaha/accounts payable), dan penyusunan pelaporan berkala. Pada usaha toko buku grosir, keberadaan sistem informasi menjadi sangat krusial karena tingginya jumlah judul buku yang dibeli dalam satu transaksi serta banyaknya supplier atau penerbit yang bekerja sama dengan toko.

Laravel digunakan sebagai framework pengembangan utama karena menganut pola arsitektur Model-View-Controller (MVC) yang memisahkan logika bisnis, representasi data, dan tampilan antarmuka secara rapi. Laravel menyediakan fitur Migration untuk version control skema database, Eloquent ORM untuk manipulasi data tanpa query SQL mentah, dan fitur DB::transaction. Fitur transaksi database ini memastikan sifat Atomicity (seluruh instruksi sukses atau batal sama sekali) terpelihara ketika sistem melakukan manipulasi data di beberapa tabel yang saling terkait, misalnya saat pembentukan hutang baru bersamaan dengan penyimpanan transaksi pembelian.

Dalam basis data relasional, konsistensi data dijamin melalui penerapan referential integrity menggunakan foreign key constraint. Hubungan entitas harus dirancang secara cermat; misalnya, mencegah penghapusan data master supplier apabila data tersebut telah memiliki keterkaitan dengan transaksi pembelian atau hutang yang masih berjalan (Restrict Constraint).

2.2 Gambaran Umum Sistem

Aplikasi GrosirBuku merupakan sistem informasi berbasis web yang dikembangkan khusus untuk menangani proses bisnis pembelian dan pengelolaan hutang pada toko buku grosir. Sistem ini memetakan seluruh entitas bisnis, mulai dari data supplier/penerbit, katalog buku, riwayat staf pembelian (user), hingga transaksi keuangan hutang, ke dalam tabel-tabel database yang saling berelasi.

Modul pembayaran hutang menjadi pusat operasional bagian keuangan toko. Proses diawali dengan transaksi pembelian buku dari supplier. Ketika staf pembelian mencatat transaksi dengan metode pembayaran kredit (tempo), sistem secara otomatis membentuk satu baris data hutang baru dengan nilai total hutang sesuai total pembelian. Pembayaran hutang kemudian dapat dilakukan secara bertahap (cicilan) melalui modul pembayaran hutang.

Setiap kali staf keuangan mencatat pembayaran, sistem akan menyimpan riwayat pembayaran, mengurangi nilai sisa hutang, dan memperbarui status hutang menjadi 'lunas' secara otomatis apabila sisa hutang telah mencapai nol. Dengan mekanisme ini, toko dapat memantau posisi keuangan terhadap seluruh supplier secara real-time.

2.3 Permasalahan yang Dihadapi

Berdasarkan analisis operasional pada usaha toko buku grosir konvensional, permasalahan utama yang dihadapi meliputi:

1. Kerentanan Human Error: Pencatatan manual nota pembelian dan cicilan pembayaran sering mengalami kesalahan hitung sisa hutang.
2. Kekacauan Manajemen Hutang: Hutang kepada banyak supplier yang dicatat pada buku catatan terpisah menyulitkan staf keuangan untuk melacak daftar hutang yang mendekati tanggal jatuh tempo.
3. Inkonsistensi Nilai Transaksi Historis: Jika harga buku pada tabel master diubah, maka nota pembelian masa lalu yang mengambil referensi harga secara dinamis akan ikut berubah, berakibat pada kacaunya laporan hutang dan keuangan bulanan.
4. Keterlambatan Pembayaran: Ketidadaan pengingat otomatis terhadap tanggal jatuh tempo hutang sering menyebabkan toko terlambat membayar sehingga berisiko merusak hubungan dengan supplier.

2.4 Solusi yang Ditawarkan Sistem

Sebagai pemecahan masalah di atas, sistem GrosirBuku menawarkan solusi berupa:

1. Modul Transaksi Terintegrasi Database: Memisahkan data header pembelian dengan rincian item buku belanjaan untuk menghasilkan struktur data yang rapi dan terukur.
2. Otomatisasi Logika Finansial: Menggunakan relasi Eloquent untuk menghubungkan pembelian kredit dengan modul hutang supplier secara otomatis.
3. Mekanisme Snapshot Harga: Menyalin nilai harga beli buku saat transaksi dilakukan langsung ke dalam kolom tabel detail pembelian, memproteksi data historis keuangan dari perubahan harga master data di masa depan.
4. Kartu Pembayaran Hutang: Mencatat setiap cicilan pembayaran ke dalam tabel riwayat, sehingga sisa hutang dan status pelunasan selalu ter-update secara otomatis.
5. Proteksi Database Transaction: Membungkus proses penyimpanan data multi-tabel (pembelian, detail, dan hutang) dalam blok instruksi transaksi database guna menjamin keamanan data dari kegagalan parsial.

2.5 Proses Pengembangan Setiap Minggu

Pengembangan modul pembayaran hutang dilaksanakan secara bertahap setiap minggu mengikuti tahapan berikut:

Minggu 2: Rancangan Sistem Pembelian dan Hutang

Fokus pada perancangan konseptual. Menghasilkan spesifikasi aturan bisnis, skema tabel basis data inti (suppliers, pembelians, detail_pembelians, hutangs, dan pembayaran_hutangs), struktur relasi kardinalitas antar entitas, serta rancangan diagram ERD sebagai panduan utama pengembangan.

Minggu 3: Implementasi Database dan Model

Mentransformasikan rancangan konseptual menjadi skema fisik melalui pembuatan berkas migration database di Laravel. Dilanjutkan dengan membangun kelas-kelas Model Eloquent (seperti Hutang dan PembayaranHutang) lengkap dengan deklarasi properti primary key khusus serta fungsi relasi relasional basis data.

Minggu 4: Analisis Controller

Membangun dan menganalisis arsitektur logika bisnis pada HutangController dan PembayaranHutangController. Menyusun alur kerja terstruktur untuk method CRUD standar, merancang alur transaksi data multi-tabel di dalam method store(), menerapkan kontrol transaksional database, serta memetakan kelebihan dan area yang membutuhkan peningkatan performa.

Minggu 5: Integrasi dan Evaluasi

Melakukan pengujian penggabungan (integration testing) antarmuka web (UI) dengan fungsi backend. Memastikan modul pembelian dan pembayaran hutang dapat bertukar data tanpa error. Mengadakan evaluasi menyeluruh terhadap rancangan kode program untuk mengidentifikasi keterbatasan sistem seperti belum adanya concurrency control dan efisiensi loop query.

BAB III

METODE PENELITIAN DAN IMPLEMENTASI SISTEM

3.1 Metode Pengembangan Sistem

Metode pengembangan sistem yang digunakan dalam penelitian ini adalah Waterfall. Metode ini dipilih karena pengembangan dilakukan secara berurutan dan bertahap, mulai dari analisis kebutuhan, perancangan sistem, implementasi, pengujian, hingga evaluasi akhir. Model ini sesuai dengan proyek yang memiliki kebutuhan cukup jelas sejak awal.

Tahapan Waterfall pada proyek ini terlihat dari proses pengerjaan per minggu. Minggu kedua digunakan untuk menyusun aturan bisnis dan rancangan database. Minggu ketiga digunakan untuk implementasi migration dan model. Minggu keempat digunakan untuk analisis controller. Minggu kelima digunakan untuk integrasi dan evaluasi keseluruhan sistem.

Keunggulan metode Waterfall adalah alurnya mudah dipahami, dokumentasinya rapi, dan hasil setiap tahap dapat dievaluasi terlebih dahulu sebelum melanjutkan ke tahap berikutnya. Hal ini sangat cocok untuk laporan akademik yang membutuhkan urutan pengerjaan yang jelas.

3.2 Objek Penelitian

Objek penelitian spesifik dalam laporan ini adalah fungsionalitas Modul Pembayaran Hutang yang berjalan di dalam ekosistem aplikasi GrosirBuku. Lingkup pengamatan diarahkan pada efektivitas alur pertukaran data antar tabel relasional, keakuratan pembentukan hutang otomatis, pencatatan cicilan pembayaran, dan pembaruan status pelunasan hutang.

3.3 Metode Pengumpulan Data

Metode pengumpulan data yang digunakan dalam penelitian ini meliputi:

1. Studi Literatur (Library Research), mempelajari buku teks, jurnal, dan dokumentasi resmi Laravel mengenai arsitektur MVC, basis data relasional, serta integritas transaksi keuangan.
2. Analisis Kebutuhan Bisnis (Business Requirement Analysis), menganalisis proses bisnis pembelian dan pembayaran hutang pada usaha toko buku grosir untuk menyelaraskan rancangan sistem dengan kebutuhan operasional nyata.
3. Perancangan dan Uji Coba Kode Program (Design and Code Prototyping), merancang skema database dan menyusun kode migration, model, serta controller untuk menganalisis relasi Eloquent, fungsi controller, dan otomatisasi data transaksi.

Dengan metode tersebut, informasi yang diperoleh menjadi lebih lengkap dan sesuai dengan kondisi implementasi sistem.

3.4 Analisis Kebutuhan Sistem

3.4.1 Kebutuhan Fungsional

1. Sistem harus mampu merekam data utama pembelian (header) yang meliputi identitas supplier, tanggal transaksi, status pembayaran, dan staf yang bertugas.

2. Sistem harus dapat menampung banyak item buku belanja ke dalam daftar detail pembelian.
3. Sistem harus mengunci harga beli buku saat transaksi sebagai nilai snapshot.
4. Sistem wajib membuat kartu hutang baru secara otomatis berstatus 'belum_lunas' dengan sisa hutang sesuai total pembelian jika staf memilih metode pembayaran credit.
5. Sistem harus dapat merekam setiap transaksi pembayaran hutang (cicilan) dan mengurangi sisa hutang secara otomatis.
6. Sistem harus dapat memperbarui status hutang menjadi 'lunas' secara otomatis ketika sisa hutang mencapai nol.
7. Sistem harus dapat menampilkan daftar hutang yang mendekati atau telah melewati tanggal jatuh tempo.

3.4.2 Kebutuhan Nonfungsional

1. Usability: Antarmuka formulir pencatatan pembayaran hutang harus sederhana, responsif, dan mudah dipahami tanpa memerlukan keahlian IT tingkat tinggi.
2. Data Integrity: Sistem wajib menjamin integritas data (nilai sisa hutang harus selalu konsisten dengan akumulasi seluruh pembayaran yang tercatat).
3. Performance: Waktu yang dibutuhkan sistem untuk memproses dan menyimpan data pembayaran hutang ke database tidak boleh melebihi 3 detik.

3.5 Perancangan Sistem

3.5.1 Aturan Bisnis

1. Transaksi pembelian dan pembayaran hutang hanya dapat dilayani oleh user yang telah terautentikasi ke dalam sistem.
2. Satu nomor nota transaksi pembelian berhak memiliki satu atau banyak item detail buku belanja.
3. Nominal subtotal pada detail pembelian dihitung melalui rumus: $\text{subtotal} = \text{jumlah_beli} \times \text{harga_beli}$.
4. Akumulasi total pembelian (total_beli) diperoleh dari penjumlahan seluruh subtotal item terkait.
5. Setiap transaksi pembelian kredit hanya boleh memicu pembentukan satu kartu hutang (relasi one-to-one).
6. Nilai sisa_hutang dihitung dari total_hutang dikurangi akumulasi seluruh jumlah_bayar pada tabel pembayaran_hutangs yang terkait.
7. Dokumen hutang memiliki jangka waktu jatuh tempo standar selama 30 hari sejak tanggal transaksi pembelian dicatat secara kredit.
8. Jumlah pembayaran hutang yang diinput tidak boleh melebihi sisa hutang yang masih berjalan.

3.5.2 Struktur Database Modul Pembayaran Hutang

Berikut adalah struktur fisik dari rancangan tabel-tabel utama yang menangani modul pembayaran hutang di dalam database.

Tabel 3.1 Skema Struktur Tabel suppliers

Field Name	Data Type	Constraint/Attribute	Keterangan
id_supplier	Bigint	Primary Key, Auto Increment	Kunci utama supplier
nama_supplier	String	Not Null	Nama supplier/penerbit
alamat	Text	Nullable	Alamat supplier
no_telp	String	Nullable	Kontak telepon supplier

Tabel 3.2 Skema Struktur Tabel pembelians

Field Name	Data Type	Constraint/Attribute	Keterangan
id_pembelian	Bigint	Primary Key, Auto Increment	Kunci utama transaksi
id_supplier	Bigint	Foreign Key (suppliers), Restrict	Relasi data supplier
id_user	Bigint	Foreign Key (users), Restrict	Relasi staf pembelian
tgl_pembelian	Date	Not Null	Tanggal transaksi dilakukan
status_bayar	Enum	['cash','credit'], Not Null	Metode pembayaran
total_beli	Integer	Default: 0	Nilai total nominal transaksi

Tabel 3.3 Skema Struktur Tabel detail_pembelians

Field Name	Data Type	Constraint/Attribute	Keterangan
id	Bigint	Primary Key, Auto Increment	Kunci baris detail
id_pembelian	Bigint	Foreign Key (pembelians), Cascade	Terikat dengan header transaksi
id_buku	Bigint	Foreign Key (bukus), Restrict	Kaitan dengan master buku
jumlah_beli	Integer	Not Null	Kuantitas buku yang dibeli
harga_beli	Integer	Not Null	Snapshot harga beli saat itu
subtotal	Integer	Not Null	Hasil kali jumlah dan harga

Tabel 3.4 Skema Struktur Tabel hutangs

Field Name	Data Type	Constraint/Attribute	Keterangan
id_hutang	Bigint	Primary Key, Auto Increment	Kunci utama kartu hutang
id_pembelian	Bigint	Foreign Key (pembelians), Unique, Restrict	Referensi nota transaksi kredit
id_supplier	Bigint	Foreign Key (suppliers), Restrict	Referensi pemilik hutang

Field Name	Data Type	Constraint/Attribute	Keterangan
total_hutang	Decimal(15,2)	Not Null	Total hutang awal (nominal transaksi)
sisahutang	Decimal(15,2)	Not Null	Sisa hutang yang belum dibayar
jatuh_tempo	Date	Not Null	Batas akhir pembayaran (30 hari)
status	Enum	['belum_lunas','lunas'], Not Null	Status pelunasan hutang

Tabel 3.5 Skema Struktur Tabel pembayaran_hutangs

Field Name	Data Type	Constraint/Attribute	Keterangan
id_pembayaran	Bigint	Primary Key, Auto Increment	Kunci baris pembayaran
id_hutang	Bigint	Foreign Key (hutangs), Restrict	Terikat dengan kartu hutang
id_user	Bigint	Foreign Key (users), Restrict	Staf yang memproses pembayaran
tgl_bayar	Date	Not Null	Tanggal pembayaran dilakukan
jumlah_bayar	Decimal(15,2)	Not Null	Nominal cicilan yang dibayarkan
metode_bayar	Enum	['tunai','transfer'], Not Null	Metode pembayaran ke supplier
keterangan	String	Nullable	Catatan tambahan pembayaran

Masing-masing tabel memiliki fungsi spesifik dan saling terhubung melalui foreign key. Hal ini membuat data transaksi lebih mudah dikelola dan tidak bercampur dengan data master.

3.5.3 Relasi Antar Entitas

Kardinalitas hubungan antar entitas di dalam database dikelola dengan aturan berikut:

- Supplier (1) → (N) Pembelian: Satu supplier dapat memasok buku dalam banyak transaksi pembelian.
- Pembelian (1) → (N) Detail Pembelian: Sebuah transaksi pembelian dapat menampung daftar rincian buku yang bervariasi.
- Pembelian (1) → (1) Hutang: Khusus transaksi kredit, record pembelian hanya akan memicu pembentukan satu baris record hutang.
- Hutang (1) → (N) Pembayaran Hutang: Satu kartu hutang dapat dilunasi melalui beberapa kali transaksi cicilan pembayaran.
- Buku (1) → (N) Detail Pembelian: Satu judul buku dapat dibeli dalam berbagai transaksi yang berbeda.

3.5.4 Alur Proses Bisnis

Alur bisnis sistem dimulai ketika staf pembelian mencatat transaksi pembelian buku dari supplier. Buku yang dibeli dimasukkan ke detail pembelian, kemudian sistem menghitung total. Setelah itu, staf memilih metode pembayaran cash atau credit. Jika cash, transaksi selesai tanpa mencatat hutang. Jika credit, sistem otomatis membentuk kartu hutang baru dengan sisa hutang sama dengan total pembelian dan jatuh tempo 30 hari ke depan.

Selanjutnya, staf keuangan dapat mencatat pembayaran hutang secara bertahap melalui modul pembayaran hutang. Setiap kali pembayaran dicatat, sistem menghitung ulang sisa hutang berdasarkan akumulasi seluruh pembayaran yang tercatat, kemudian memperbarui status hutang menjadi 'lunas' apabila sisa hutang telah mencapai nol.

3.5.5 Perancangan Antarmuka

Perancangan antarmuka pada tahap ini disajikan dalam bentuk sketsa (wireframe) rancangan tampilan sebagai acuan pengembangan tampilan aplikasi yang sesungguhnya.

Data Hutang

No	Supplier	Total Hutang	Sisa Hutang	Jatuh Tempo	Status	Aksi
1	CV Ilmu Cerdas	Rp 3.000.000	Rp 1.200.000	15 Agu 2026	Belum Lunas	[Detail]
2	Penerbit Kita Baca	Rp 1.500.000	Rp 0	02 Agu 2026	Lunas	[Detail]
[+ Tambah Pembayaran]		[Filter Status]	[Filter Jatuh Tempo]			

Gambar 3.1 Sketsa Halaman Data Hutang

Halaman Data Hutang berfungsi untuk menampilkan seluruh daftar hutang toko kepada supplier. Pada halaman ini pengguna dapat melihat nama supplier, total hutang, sisa hutang, tanggal jatuh tempo, serta status pelunasan. Tersedia pula menu aksi untuk melihat detail dan riwayat pembayaran setiap hutang.

Detail Hutang & Riwayat Pembayaran

Supplier	: CV Ilmu Cerdas	Status	: Belum Lunas
Total Hutang	: Rp 3.000.000		
Sisa Hutang	: Rp 1.200.000		
Jatuh Tempo	: 15 Agu 2026		
Progress Bayar	: [#####----] 60% terbayar		
Riwayat Pembayaran:			
- Rp 1.000.000	05 Jul 2026	Transfer	
- Rp 800.000	12 Jul 2026	Tunai	
[Tambah Pembayaran]			

Gambar 3.2 Sketsa Halaman Detail Hutang & Riwayat Pembayaran

Halaman Detail Hutang menampilkan informasi lengkap mengenai satu kartu hutang tertentu, termasuk progres pembayaran dan riwayat cicilan yang telah dilakukan. Halaman ini memudahkan staf keuangan dalam memantau hutang yang masih berjalan sekaligus menjadi acuan dalam mencatat pembayaran berikutnya.

Form Pembayaran Hutang

Hutang	: #HT-0001 - CV Ilmu Cerdas
Sisa Hutang	: Rp 1.200.000

Tanggal Bayar*	: [20/07/2026]
Jumlah Bayar*	: [Rp _____]
Metode Bayar*	: () Tunai () Transfer
Keterangan	: [_____]
[Simpan Pembayaran] [Batal]	

Gambar 3.3 Sketsa Halaman Form Pembayaran Hutang

Halaman Form Pembayaran Hutang digunakan oleh staf keuangan untuk mencatat cicilan pembayaran ke supplier. Sistem akan memvalidasi agar jumlah bayar tidak melebihi sisa hutang, kemudian secara otomatis memperbarui sisa hutang dan status kartu hutang setelah data disimpan.

Data Pembelian (Credit)

No	Tanggal	Supplier	Total	Jenis	Aksi
1	01 Jul 2026	CV Ilmu Cerdas	Rp 3.000.000	Credit	[Detail]
2	10 Jul 2026	Penerbit Kita Baca	Rp 1.500.000	Credit	[Detail]
3	15 Jul 2026	Toko Distribusi Buku	Rp 800.000	Cash	[Detail]

Gambar 3.4 Sketsa Halaman Data Pembelian (Credit)

Halaman Data Pembelian menampilkan seluruh transaksi pembelian buku dari supplier, baik yang dibayar tunai maupun kredit. Transaksi berjenis credit akan otomatis terhubung dengan data hutang pada modul pembayaran hutang.

Notifikasi Hutang Jatuh Tempo

! Perhatian: 2 hutang akan jatuh tempo dalam 7 hari
- CV Ilmu Cerdas : Rp 1.200.000 (jatuh tempo 3 hari lagi)
- Penerbit Sinar Ilmu : Rp 500.000 (jatuh tempo 6 hari lagi)
[Lihat Semua Hutang]

Gambar 3.5 Sketsa Halaman Notifikasi Hutang Jatuh Tempo

Halaman notifikasi ini membantu staf keuangan agar tidak melewatkan tanggal jatuh tempo pembayaran hutang, sehingga hubungan baik dengan supplier tetap terjaga.

BAB IV

IMPLEMENTASI DAN EVALUASI SISTEM

4.1 Implementasi Database (Migration)

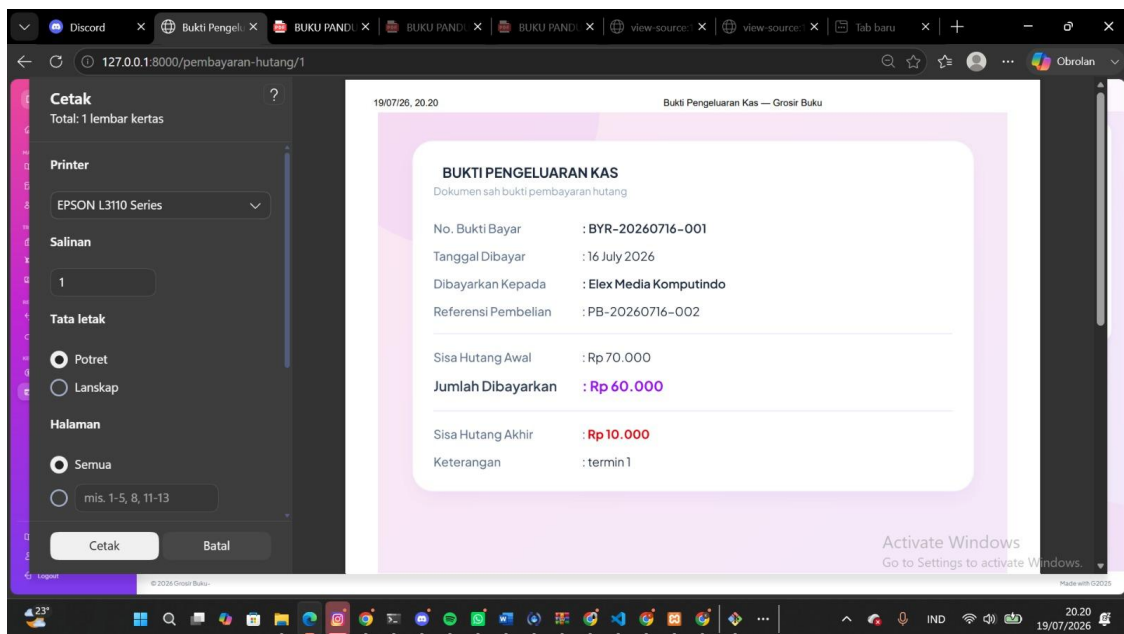
Skema database dibangun menggunakan file migrasi Laravel untuk menjamin standardisasi tipe data kolom antar komputer pengembang. Untuk memverifikasi bahwa skema fisik yang berjalan di MySQL sudah sesuai dengan rancangan pada Tabel 3.1 sampai 3.5, berikut disertakan pula tangkapan layar (screenshot) struktur tabel hasil migrasi yang dilihat melalui phpMyAdmin.

A. Tabel suppliers

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('suppliers', function (Blueprint $table) {
            $table->id('id_supplier');
            $table->string('nama_supplier');
            $table->text('alamat')->nullable();
            $table->string('no_telp')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('suppliers');
    }
};
```



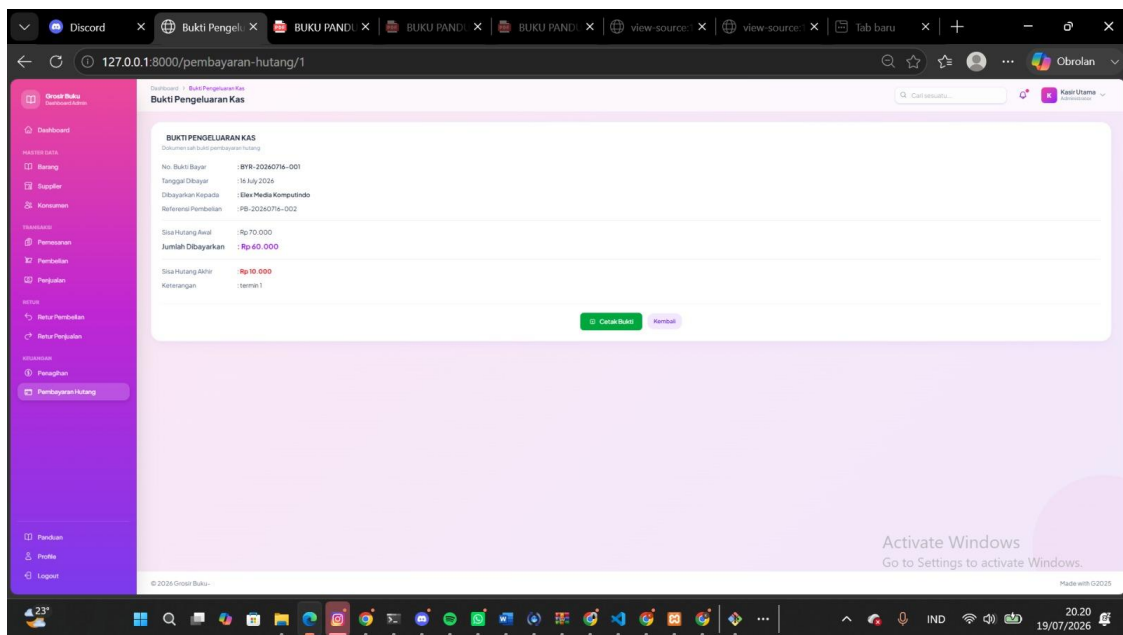
Gambar 4.1 Struktur Tabel suppliers pada phpMyAdmin

B. Tabel pembelians

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('pembelians', function (Blueprint $table) {
            $table->id('id_pembelian');
            $table->foreignId('id_supplier')
                ->constrained('suppliers', 'id_supplier')
                ->onDelete('restrict');
            $table->foreignId('id_user')
                ->constrained('users', 'id_user')
                ->onDelete('restrict');
            $table->date('tgl_pembelian');
            $table->enum('status_bayar', ['cash', 'credit']);
            $table->integer('total_beli')->default(0);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('pembelians');
    }
};
```



Gambar 4.2 Struktur Tabel pembelians pada phpMyAdmin

C. Tabel detail_pembelians

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```

return new class extends Migration {
    public function up(): void
    {
        Schema::create('detail_pembelians', function (Blueprint $table) {
            $table->id();
            $table->foreignId('id_pembelian')
                ->constrained('pembelians', 'id_pembelian')
                ->onDelete('cascade');
            $table->foreignId('id_buku')
                ->constrained('bukus', 'id_buku')
                ->onDelete('restrict');
            $table->integer('jumlah_beli');
            $table->integer('harga_beli'); // snapshot harga saat transaksi
            $table->integer('subtotal');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('detail_pembelians');
    }
};

```

D. Tabel hutangs

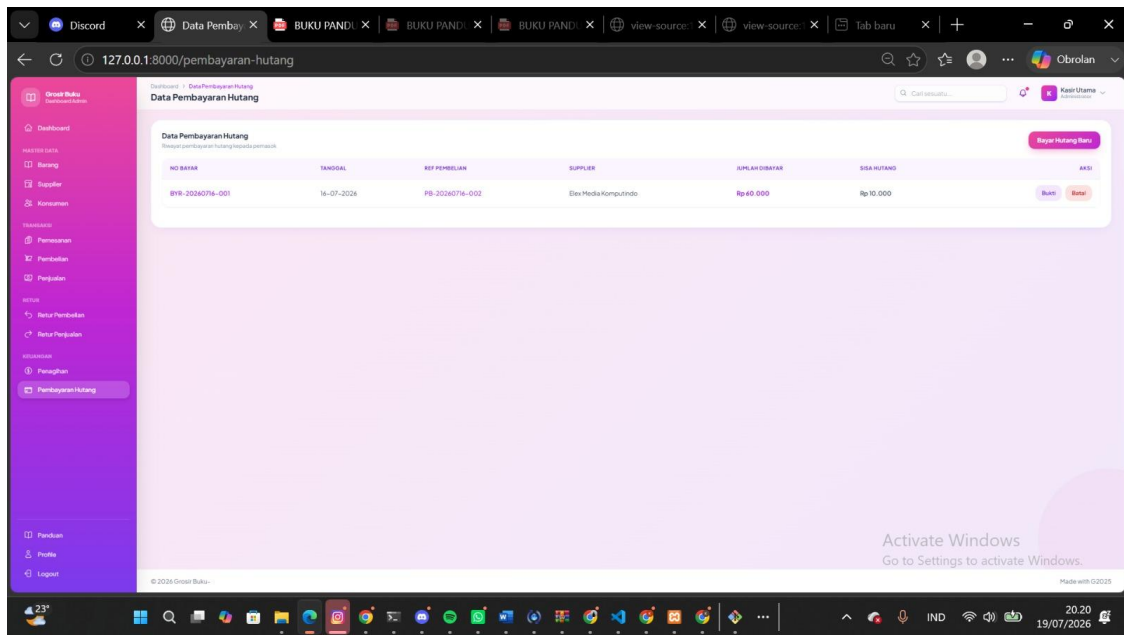
```

<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('hutangs', function (Blueprint $table) {
            $table->id('id_hutang');
            $table->foreignId('id_pembelian')
                ->unique()
                ->constrained('pembelians', 'id_pembelian')
                ->onDelete('restrict');
            $table->foreignId('id_supplier')
                ->constrained('suppliers', 'id_supplier')
                ->onDelete('restrict');
            $table->decimal('total_hutang', 15, 2);
            $table->decimal('sisahutang', 15, 2);
            $table->date('jatuh_tempo');
            $table->enum('status', ['belum_lunas', 'lunas'])->default('belum_lunas');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('hutangs');
    }
};

```



Gambar 4.3 Struktur Tabel hutangs pada phpMyAdmin

E. Tabel pembayaran_hutangs

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('pembayaran_hutangs', function (Blueprint $table) {
            $table->id('id_pembayaran');
            $table->foreignId('id_hutang')
                ->constrained('hutangs', 'id_hutang')
                ->onDelete('restrict');
            $table->foreignId('id_user')
                ->constrained('users', 'id_user')
                ->onDelete('restrict');
            $table->date('tgl_bayar');
            $table->decimal('jumlah_bayar', 15, 2);
            $table->enum('metode_bayar', ['tunai', 'transfer']);
            $table->string('keterangan')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('pembayaran_hutangs');
    }
};
```

The screenshot displays a web application interface for managing supplier debt payments. The main content area is titled 'Form Pembayaran Hutang Supplier'. It contains several input fields: 'NO. BUKTI BAYAR' with the value 'BYR-20260719-001', 'TANGGAL PEMBAYARAN' with the value '19/07/2026', 'PELITRANGGASU PEMBELIAN KREDIT' with a dropdown menu showing '--- Pilih Nomor Pembelian ---', 'NAMA SUPPLIER', 'TOTAL SISA HUTANG (Rp)', and 'JUMLAH BAYAR KESUKSESAN (Rp)'. Below these fields is a text area for 'KETERANGAN / CATATAN' with the example text 'Contoh: Pembayaran termi 1 Lunas via Bank Mandiri'. At the bottom of the form are two buttons: 'Proses Pembayaran' and 'Batal'. The left sidebar is a navigation menu with a pink header 'Grosir Buku' and various menu items including 'Dashboard', 'Master Data', 'Supplier', 'Konsumen', 'Pembelian', 'Pengiriman', and 'Laporan'. The bottom status bar shows the date '19/07/2026' and time '20:20'.

Gambar 4.4 Struktur Tabel pembayaran_hutangs pada phpMyAdmin

Berdasarkan keempat gambar di atas, struktur fisik tabel yang berhasil dijalankan di MySQL melalui phpMyAdmin telah sesuai dengan rancangan skema pada Tabel 3.1, 3.2, 3.4, dan 3.5, baik dari sisi nama kolom, tipe data, maupun kolom timestamps (created_at, updated_at) yang ditambahkan otomatis oleh Laravel.

4.2 Implementasi Model Eloquent

Model bertindak sebagai jembatan interaksi objek berorientasi dengan tabel relasional di database.

A. Model Supplier

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Supplier extends Model
{
    protected $primaryKey = 'id_supplier';
    protected $fillable = ['nama_supplier', 'alamat', 'no_telp'];

    public function pembelians()
    {
        return $this->hasMany(Pembelian::class, 'id_supplier');
    }

    public function hutangs()
    {
        return $this->hasMany(Hutang::class, 'id_supplier');
    }
}
```

B. Model Pembelian

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Pembelian extends Model
{
    protected $primaryKey = 'id_pembelian';
    protected $fillable = [
        'id_supplier', 'id_user', 'tgl_pembelian',
        'total_beli', 'status_bayar',
    ];

    public function supplier()
    {
        return $this->belongsTo(Supplier::class, 'id_supplier', 'id_supplier');
    }

    public function user()
    {
        return $this->belongsTo(User::class, 'id_user', 'id_user');
    }

    public function detailPembelians()
    {
        return $this->hasMany(DetailPembelian::class, 'id_pembelian');
    }

    public function hutang()
    {
        return $this->hasOne(Hutang::class, 'id_pembelian');
    }
}
```

```

    }
}

```

C. Model Hutang

```

<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Hutang extends Model
{
    protected $primaryKey = 'id_hutang';
    protected $fillable = [
        'id_pembelian', 'id_supplier', 'total_hutang',
        'sisahutang', 'jatuh_tempo', 'status',
    ];

    protected $casts = [
        'total_hutang' => 'decimal:2',
        'sisahutang' => 'decimal:2',
        'jatuh_tempo' => 'date',
    ];

    public function supplier()
    {
        return $this->belongsTo(Supplier::class, 'id_supplier', 'id_supplier');
    }

    public function pembelian()
    {
        return $this->belongsTo(Pembelian::class, 'id_pembelian', 'id_pembelian');
    }

    public function pembayaranHutangs()
    {
        return $this->hasMany(PembayaranHutang::class, 'id_hutang', 'id_hutang');
    }

    // Hitung ulang sisa_hutang & status setiap ada perubahan pembayaran.
    public function updateSisaHutang(): void
    {
        $totalDibayar = $this->pembayaranHutangs()->sum('jumlah_bayar');
        $sisahutang = $this->total_hutang - $totalDibayar;

        $this->sisahutang = max($sisahutang, 0);
        $this->status = $this->sisahutang <= 0 ? 'lunas' : 'belum_lunas';
        $this->save();
    }
}

```

D. Model PembayaranHutang

```

<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class PembayaranHutang extends Model
{
    protected $primaryKey = 'id_pembayaran';
}

```

```
protected $fillable = [
    'id_hutang', 'id_user', 'tgl_bayar',
    'jumlah_bayar', 'metode_bayar', 'keterangan',
];

protected $casts = [
    'jumlah_bayar' => 'decimal:2',
    'tgl_bayar' => 'date',
];

public function hutang()
{
    return $this->belongsTo(Hutang::class, 'id_hutang', 'id_hutang');
}

public function user()
{
    return $this->belongsTo(User::class, 'id_user', 'id_user');
}
}
```


4.3 Implementasi Controller

PembayaranHutangController bertindak sebagai pengatur alur data (traffic controller) yang menjembatani request antarmuka staf keuangan dengan operasi basis data.

Tabel 4.1 Pemetaan Fungsi Method pada PembayaranHutangController

Method	HTTP Verb	URL Route	Implementasi Fungsi Logika
index()	GET	/hutang	Menampilkan seluruh daftar kartu hutang dengan optimasi Eager Loading with().
show()	GET	/hutang/{id}	Menampilkan detail satu kartu hutang beserta riwayat pembayarannya.
store()	POST	/hutang/{id}/bayar	Menyimpan transaksi pembayaran hutang baru dan memperbarui sisa hutang dalam kendali DB::beginTransaction().
destroy()	DELETE	/pembayaran-hutang/{id}	Membatalkan data pembayaran dan memicu perhitungan ulang sisa hutang terkait.

Berikut potongan implementasi inti logika pada method store() di dalam PembayaranHutangController.php:

```
public function store(Request $request, Hutang $hutang)
{
    $request->validate([
        'tgl_bayar' => 'required|date',
        'jumlah_bayar' => 'required|numeric|min:1',
        'metode_bayar' => 'required|in:tunai,transfer',
        'keterangan' => 'nullable|string|max:255',
    ]);

    // Validasi: jumlah bayar tidak boleh melebihi sisa hutang berjalan
    if ($request->jumlah_bayar > $hutang->sisa_hutang) {
        return back()->withErrors([
            'jumlah_bayar' => 'Jumlah bayar melebihi sisa hutang yang berjalan.',
        ]);
    }

    DB::transaction(function () use ($request, $hutang) {
        // Simpan riwayat pembayaran
        PembayaranHutang::create([
            'id_hutang' => $hutang->id_hutang,
            'id_user' => Auth::user()->id_user,
            'tgl_bayar' => $request->tgl_bayar,
            'jumlah_bayar' => $request->jumlah_bayar,
            'metode_bayar' => $request->metode_bayar,
            'keterangan' => $request->keterangan,
        ]);

        // Hitung ulang sisa hutang & perbarui status pelunasan
        $hutang->updateSisaHutang();
    });

    return redirect()->route('hutang.show', $hutang->id_hutang)
        ->with('success', 'Pembayaran hutang berhasil disimpan!');
}
```

4.4 Evaluasi Implementasi

Berdasarkan hasil analisis rancangan sistem dan struktur kode program, kelebihan dan kekurangan sistem diidentifikasi secara jujur sebagai berikut:

A. Kelebihan Sistem

- a. Keamanan Transaksi Finansial: Penerapan instruksi `DB::beginTransaction()` mencegah munculnya data menggantung akibat kegagalan simpan parsial antara riwayat pembayaran dan pembaruan sisa hutang.
- b. Kekebalan Histori Harga: Konsep penyalinan harga beli ke kolom snapshot detail pembelian menjamin keakuratan laporan hutang dari fluktuasi harga master data.
- c. Validasi Batas Pembayaran: Pengecekan `jumlah_bayar` terhadap `sisa_hutang` sebelum data disimpan mencegah kesalahan input yang membuat sisa hutang bernilai negatif.
- d. Otomatisasi Status Pelunasan: Fungsi `updateSisaHutang()` secara konsisten memperbarui status hutang tanpa perlu intervensi manual dari staf.

B. Kekurangan / Kelemahan Sistem Saat Ini

- a. Ketidaktersediaan Concurrency Control: Fungsi penyimpanan pembayaran belum mengimplementasikan metode pessimistic locking (`lockForUpdate()`), sehingga berpotensi menimbulkan race condition apabila dua staf mencatat pembayaran untuk hutang yang sama pada saat bersamaan.
- b. Belum Ada Notifikasi Otomatis: Sistem belum memiliki mekanisme pengingat (reminder) otomatis melalui email atau notifikasi aplikasi terhadap hutang yang mendekati tanggal jatuh tempo.
- c. Problem Skalabilitas Data Dropdown: Fungsi yang memuat seluruh data hutang aktif menggunakan perintah query tanpa pagination berpotensi membebani performa apabila jumlah hutang toko sudah mencapai ribuan data.

Berdasarkan hasil evaluasi implementasi yang telah dilakukan, dapat disimpulkan bahwa modul pembayaran hutang pada aplikasi GrosirBuku telah mampu memenuhi kebutuhan utama proses pencatatan hutang dan cicilan pembayarannya kepada supplier. Sistem ini juga berhasil menjaga integritas data melalui penerapan relasi basis data dan mekanisme transaksi pada Laravel. Meskipun demikian, masih terdapat beberapa aspek yang dapat dikembangkan lebih lanjut, seperti peningkatan performa, keamanan transaksi, dan skalabilitas sistem agar mampu mendukung kebutuhan operasional yang lebih kompleks di masa mendatang.

DAFTAR PUSTAKA

- Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed.). Pearson.
- Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson.
- Enterprise, J. (2022). Belajar Otodidak Laravel 9. Jakarta: PT Elex Media Komputindo.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Kadir, A. (2020). Dasar Perancangan dan Implementasi Database Relasional. Yogyakarta: Andi.
- Kristanto, A. (2018). Perancangan Sistem Informasi dan Aplikasinya. Yogyakarta: Gava Media.
- Laravel. (2026). Laravel Documentation. <https://laravel.com/docs>
- MySQL. (2026). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/>
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- Rosa, A. S., & Shalahuddin, M. (2018). Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek (Edisi Revisi). Bandung: Informatika.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.